

# Formal verification of *Algorithms for triple-word arithmetic* verified results, and where they differ from the paper

Laurent Théry

Inria laurent.thery@inria.fr

29 July 2026

This note summarises a complete machine-checked verification of the paper, and lists every point where the formal proof departs from the printed text. It uses the paper’s own notation throughout; no knowledge of the proof assistant is needed to read it.

Everything below has been checked by machine. Counterexamples are exact and reproducible; where a constant differs, both values are proved.

## 1 What has been verified

Algorithms 1–15, together with the appendix Algorithms 18 and 20, and Theorems 1–11. Radix 2, precision  $p$ , round to nearest ties-to-even,  $u = 2^{-p}$ .

|                                 | published                                      | verified                                              |
|---------------------------------|------------------------------------------------|-------------------------------------------------------|
| Thm 1, Cor. 1 ( <i>VecSum</i> ) | —                                              | as stated                                             |
| Thm 2 ( <i>VSEB</i> )           | —                                              | as stated                                             |
| Thm 3 (keep first $k$ )         | $2u^k + 4.2u^{k+1}$                            | as stated, for general $k$                            |
| Thm 4 ( <i>ToTW</i> )           | $p \geq 4$                                     | as stated                                             |
| Thm 5 ( <i>RoundTW</i> )        | $\text{RoundTW}(\bar{x}) = \text{RN}(\bar{x})$ | as stated, <b>after correcting Alg. 7</b> (§2.1)      |
| Thm 6 ( <i>TWSum</i> )          | $p \geq 4$                                     | as stated ( <b>its proof has a false step</b> , §2.3) |
| Thm 7 (Alg. 9)                  | $28u^3 + 107u^4$                               | as stated                                             |
| Thm 8 (Alg. 11)                 | $10.5u^3 + 39u^4$                              | as stated ( <b>gap in the case analysis</b> , §2.4)   |
| Thm 9 (Alg. 13)                 | $11.5u^3 + 1465u^4$                            | $11.5u^3 + \mathbf{1830}u^4$                          |
|                                 | $19u^3 + 1502u^4$                              | $19u^3 + \mathbf{1870}u^4$ (§2.6)                     |
| Thm 10 (Alg. 14)                | $24u^3 + 1509u^4$                              | $\mathbf{27}u^3 + \mathbf{2500}u^4$                   |
|                                 | $39u^3 + 1582u^4$                              | $\mathbf{42}u^3 + \mathbf{2575}u^4$ (§3)              |
| Thm 11 (Alg. 15)                | $24u^3 + 10260u^4$                             | as stated                                             |
|                                 | $39u^3 + 10333u^4$                             | as stated (§4)                                        |

Two of the  $u^3$  constants move (Theorem 10) and two of the  $u^4$  constants move (Theorem 9). **Every other  $u^3$  constant in the paper is confirmed exactly** — including  $11.5 = 10.5 + 1$  and  $19 = 18 + 1$ , and all of Theorem 11.

Not formalised: Algorithms 16, 17, 19 (the sign-folded variants — they save one operation each and change no bound).

## 2 Points that need a correction

### 2.1 Algorithm 7 (*RoundTW*): the inner test has the wrong polarity

As printed, the first **if** returns  $\text{RN}(x_0 + x_1)$  when

$$\text{RN}(x_0 + 2x_1) \text{ is inexact} \quad \text{or} \quad \text{RN}\left(-\left(\frac{3}{2}u - 2u^2\right) \cdot x_0\right) \neq x_1.$$

The second disjunct should be “=”, not “ $\neq$ ”.

At a genuine midpoint  $x_1 = \pm \frac{1}{2}\text{ulp}(x_0)$ , the quantity  $\text{RN}(-(\frac{3}{2}u - 2u^2)x_0)$  has magnitude  $\approx \frac{3}{2}u|x_0|$  and, for positive  $x_1$ , the opposite sign — so it never equals  $x_1$ , the printed test is true, and the algorithm returns  $\text{RN}(x_0 + x_1)$ , **ignoring  $x_2$  exactly when  $x_2$  decides the rounding**. Conversely, the paper’s own “special case”  $x_0 = 1 + 2u$ ,  $x_1 = -\frac{3}{2}u$  is precisely where  $\text{RN}(-(\frac{3}{2}u - 2u^2)x_0) = x_1$ , so the printed condition takes the directional branch there — also the wrong way round.

**Counterexample (binary64).**  $\bar{x} = (1, u, u^2)$  is a valid triple word, and  $x_0 + x_1$  is exactly the midpoint of  $[1, 1 + 2^{-52}]$ .

|                        |               |
|------------------------|---------------|
| $\text{RN}(\bar{x})$   | $1 + 2^{-52}$ |
| Algorithm 7 as printed | 1 — wrong     |
| Algorithm 7 with “=”   | $1 + 2^{-52}$ |

An exact sweep over random valid triple words fails on about 0.3% of inputs (all of them midpoints) for  $p = 4, 6, 8, 10$ . Theorem 5 is true of the corrected algorithm, and the proof text then reads correctly as written.

### 2.2 Lemma 1: two missing hypotheses

Lemma 1 ( $\frac{1}{2}\text{ulp}(x)$  divides  $\text{RN}(x + y)$ , used in the proof of Theorem 7) needs  $y$  to be a floating-point number and  $x \neq 0$ . It is false without them.

### 2.3 Theorem 6: an intermediate claim of the proof is false

The proof asserts that the output of *VecSum* is F-nonoverlapping. It is not:

$$(15, 15, \frac{15}{16}, \frac{15}{16}) \quad \text{at } p = 4$$

is a legal input whose *VecSum* output violates Definition 2. **Theorem 6 itself is true** — *VSEB* repairs the defect — but the argument has to go through *VSEB* rather than through *VecSum* alone.

### 2.4 Theorem 8: a gap in the case analysis

The case analysis splits on  $\bar{x}\bar{y} \geq \frac{3}{2} - 5u$  and  $\bar{x}\bar{y} < \frac{3}{2} - 6u$ , leaving the interval between the two uncovered. Using the single threshold  $\frac{3}{2} - 7u$  closes it, and the proof goes through unchanged. **The statement is unaffected.**

## 2.5 Section 8.3: the tail bound needs ulp, not uls

The argument that the leading limb of  $3Prod_{2,3}^{\text{acc}}(\bar{b}, \bar{x})$  equals 1 bounds the tail using F-nonoverlapping, that is, using uls. That is not enough:

$$(16, -8, -4, -2, -1)$$

is F-nonoverlapping, sums to 1, and has leading limb 16. What the argument actually needs is the ulp-**scale** separation of the two leading limbs — which the *2Sum* structure of the inner *VecSum* does supply, so the conclusion  $e_0 = 1$  stands. Only the justification has to change.

## 2.6 $\delta_2$ (Algorithm 18) is $u^3 + 620u^4$ , not $u^3 + 260u^4$

Two distinct issues.

(a) **Remark 10 is false.** Algorithm 18 uses *Fast2Sum*( $b_1, z_{01}^+$ ) without establishing the ordering, on the grounds that when the condition fails  $b_1$  is negligible. But the condition fails exactly when  $|b_1| < |z_{01}^+|$ , and the worst case  $|b_1| \approx \text{ulp}(z_{01}^+)$  loses  $\approx u|z_{01}^+| \approx 41u^3$  — not  $O(u^4)$ . Legal binary64 inputs reach **32u<sup>3</sup>**, that is, 32 times the announced  $\delta_2$ . The last line, *Fast2Sum*( $e_1, e_2$ ), has the same hole ( $|e_1| < |e_2|$  is legal, and costs  $\approx 3u^3$ ).

*This is repairable at no arithmetic cost:* sort the two arguments first. A *Fast2Sum* preceded by one test is error-free unconditionally, so Algorithm 18 keeps its 20 operations and gains 2 tests.

(b) **The  $u^4$  term.** With the repair in place we prove  $\delta_2 = u^3 + 620u^4$ . The  $u^3$  term is exactly the announced one; the  $u^4$  term is not. Consequence for Theorem 9:

|          | published           | verified            |
|----------|---------------------|---------------------|
| accurate | $11.5u^3 + 1465u^4$ | $11.5u^3 + 1830u^4$ |
| fast     | $19u^3 + 1502u^4$   | $19u^3 + 1870u^4$   |

## 2.7 The exponent range

All the results are proved with an unbounded exponent range — the model the paper works in implicitly. This is worth stating in the text, because Theorem 1 and Corollary 1 are genuinely false near the underflow threshold.

## 3 One place where our bound is weaker, and we believe the paper is right

### 3.1 $\delta_3$ (Algorithm 20) and Theorem 10

We can only prove  $\delta_3 = 6u^3 + 1250u^4$  against the announced  $3u^3 + 264u^4$ , whence Theorem 10 at  $27u^3 + 2500u^4$  and  $42u^3 + 2575u^4$  instead of  $24u^3 + 1509u^4$  and  $39u^3 + 1582u^4$ .

The reason is that a *triple* word only gives  $|x_1| < \text{ulp}(x_0)$  and  $|x_2| < \text{ulp}(x_1)$ , that is, twice a double word's separation, so the two neglected roundings cost  $2u^3$  and  $4u^3$  instead of  $1u^3$  and  $2u^3$ . The announced  $3u^3$  is what the double-word separations would give.

**We do not claim this is an error.** A numerical search over legal inputs reaches only about  $2.5u^3$ , so  $3u^3$  does look attained and the remaining factor of two is very likely slack in our proof, not in yours. We record it only so that the discrepancy is not mistaken for a disagreement.

### 3.2 A hypothesis that is implicit in Algorithm 20

Worth making explicit in the appendix:  $\delta_3$  depends on **how close the second factor is to 1**, not merely on its leading limb being 1. Algorithm 14 supplies a second factor within  $40u^2$  of 1; Algorithm 15 can only supply one within  $\approx 101u^2$ , because there the seed error enters undamped (see §4). Our bound is therefore stated as a function of that tolerance  $c$ :

$$\delta_3(c) = 6u^3 + (31c + 10)u^4 \quad \text{for a second factor within } cu^2 \text{ of 1.}$$

The  $u^3$  term does not depend on  $c$  at all.

## 4 Theorem 11 (Algorithm 15)

### 4.1 It holds exactly as published

$24u^3 + 10260u^4$  and  $39u^3 + 10333u^4$  at  $p \geq 11$ , verified — and this is the only theorem in the paper for which the printed constants survive despite *every* intermediate constant of ours being worse ( $\delta_3$  above; and we could only reach  $100u^2$  for the seed, where the supplementary material proves  $81u^2 + 622u^3$ ).

That is not luck. **Our proof takes a different route**, and has to. The supplementary material bounds  $|\delta_1 - (\delta_1 + \delta_2)/2|$  by the triangle inequality, which gives  $\delta_1$  the weight 1.5. That discards a real cancellation:  $i^{(1)}$  occurs twice — once as a factor of the result, once inside  $i^{(2)}$  — with opposite signs. Keeping it gives  $\delta_1$  the weight 1/2, and the total becomes

$$|y - \sqrt{x}| \leq \left( \frac{1}{2}\delta_1 + \frac{1}{2}\delta_2 + \delta_3 + 15200u^4 \right) \sqrt{x},$$

which is what leaves enough  $u^3$  room to absorb our worse  $u^4$  terms. Done termwise, with our  $\delta_3$ , Theorem 11 would come out at  $29u^3 / 44u^3$  and the published bound would fail on both terms at once.

This suggests that the weight 1.5 on  $\delta_1$  is simply loose, and that the true first-order sensitivity of Algorithm 15 to its first product is one half, not one and a half.

### 4.2 $p \geq 11$ is confirmed, and confirmed sharp

The supplementary material's  $9916u^4$  residual is exactly  $E^2(3 + E)/2$  evaluated at  $E = 81u^2 + 622u^3$ : that is  $9915.5u^4$  at  $p = 11$ , and  $9989.9u^4$  at  $p = 10$ , where it exceeds the stated value. So the precision requirement is real, the constant is what it is announced to be, and the two are consistent.

## 5 The reference implementation

We also read the C reference implementation of this arithmetic (`triple_float.c` from `NXP-Research/TW Falcon binary32`). It matters here because it is independent evidence about the two points above.

**The algorithms are transcribed faithfully.** `tw_sqrt` follows Algorithm 15 line for line — the  $1 + 4u$  seed, the exact halvings,  $1.5f - h_{01}^{(2)}$ , the two FMAs,  $Fast2Sum(b_{01}, b_{12})$  — and `tw_sum`, `tw_prod`, `tw_reciprocal`, `tw_div` likewise follow Algorithms 8, 9/10, 13 and 14. Algorithm 7 is not implemented at all, so the defect of §2.1 does not reach this code.

But the unguarded *Fast2Sum* of §2.6(a) is present verbatim, in both Algorithm 18 and Algorithm 20:

```
two_prod(a.x[0], i[0], &z01_plus, &z01_min);
fast_two_sum(a.x[1], z01_plus, &b0t, &b1t); /* Fast2Sum(b1, z01+) */
...
fast_two_sum(e1, e2, &r.x[1], &r.x[2]); /* and the last line */
```

Those two routines are called by the reciprocal, the division **and** the square root, so all three inherit the loss. Sorting the two arguments repairs it at no arithmetic cost.

**A second, related deviation.** Algorithms 9 and 10 read  $(e_0, \dots, e_4) \leftarrow \text{VecSum}(z_{00}^+, b_0, b_1, c, z_3)$ , and *VecSum* is built from *2Sum*; the implementation uses *Fast2Sum* at those steps instead. Same species of unproven shortcut. (The *addition* path does use *2Sum*, correctly.)

**And the source itself flags two errors in the appendix algorithms.** In the routine computing  $\frac{3}{2} - 3\text{Prod}_{2,3}^{\text{acc}}(\bar{b}', i^{(1)})$  one finds

```
// fault in algorithm specification, says z32 instead of z31 here
z3 = z31 + z01_min;
```

and, a few lines further down,

```
// Algorithms specifies * 0.5 somewhere, but For some reason the only way
// it is correct is by just doing it the normal way
```

These are the sign-folded variants (Algorithms 16, 17 and 19). We scoped them out because they are presented as purely operational refinements — they fold a  $+$  into a  $-$  inside the multiplication, saving the explicit  $2 - \cdot / \frac{3}{2} - \cdot$  and one operation, and are said to change no bound — while Theorems 9, 10 and 11 are stated about Algorithms 13, 14 and 15 as written in the body. (We did formalise the two appendix algorithms the theorems genuinely need, 18 and 20.)

That scoping now looks too generous. Algorithm 18 is an appendix algorithm too, and it carried a real defect *and* a  $u^4$  constant more than twice the announced one (§2.6). The comments above say 16/17/19 as printed do not work either. And these are the routines the implementation actually runs — so nothing in the verified chain covers the deployed reciprocal, division or square root. **The appendix is worth a re-read, and is the natural next thing to verify.**

*Scope: we read the main operation paths, not the constant-time variants or the conversion routines.*

## 6 In one line

The published statements stand, with four amendments: **Algorithm 7’s inner test must be inverted**, **Algorithm 18’s two *Fast2Sum* calls must sort their arguments**,  $\delta_2$ ’s  $u^4$  **term is 620, not 260** (hence Theorem 9’s 1830 / 1870), and **Lemma 1 needs two hypotheses**. Three further points (§2.3, §2.4 and Section 8.3’s tail bound) are gaps in proofs whose statements are correct, and Theorem 11 holds verbatim. The C reference implementation carries the *Fast2Sum* defect unrepaired, and its own comments report two further errors in the appendix algorithms (§5).